PowerShell Execution and Network Activity Analysis Using Sysmon and Splunk

Investigation date: 11 October 2026 | Simulation date: 10 October 2026

Summary

This project analyses a harmless PowerShell simulation in a virtual lab. Sysmon events collected in Splunk confirmed execution of simulation.ps1 and the creation of whoami.exe and hostname.exe child processes. A TCP connection was associated with the PowerShell PID using the host and close timing. The collected telemetry did not confirm the download of sample.txt.

Lab environment

Machine	Role	Lab IP
Windows 11	Sysmon and Splunk Universal Forwarder	192.168.50.10
Ubuntu	Splunk server running in Docker	192.168.50.30
Kali Linux	HTTP server hosting simulation files	192.168.50.20

The virtual machines communicated through a private VirtualBox lab network. Splunk Web was available on port 8000, and the forwarder sent events to port 9997. The HTTP server used port 8080.

The data source was Microsoft-Windows-Sysmon/Operational. Events were forwarded as XML into index main with host vmwin11. A whoami.exe process creation event appeared in Splunk, confirming event forwarding; this test did not verify coverage of every event type.

Scope and assistance

This was a manual investigation of a known simulation script. No automated alert was configured, and no real malware was used. Splunk documentation and AI assistance supported configuration and query development. Query results were checked against the collected events.

Collection verification

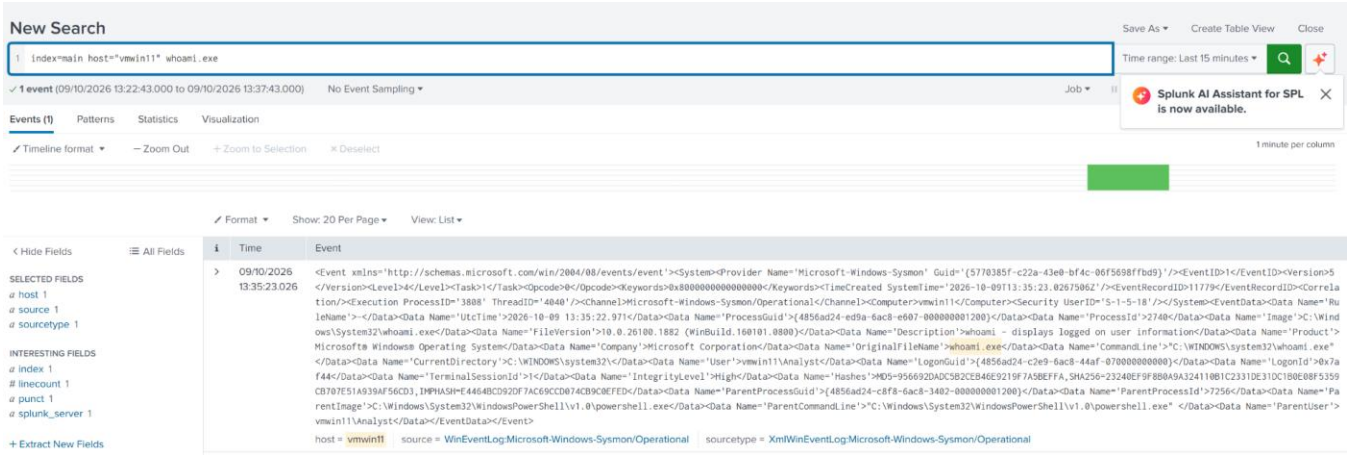


Figure 1. A whoami.exe event in Splunk confirms forwarding from vmwin11.

Simulation

The harmless script runs whoami.exe and hostname.exe, saves their output in the temporary SOC-Lab directory, and requests sample.txt from the lab HTTP server. The script and text file were hosted on Kali Linux.

```
python3 -m http.server 8080 --bind 192.168.50.20 --directory ~/soc-lab
```

This command starts Python's HTTP server on the lab IP and serves files from the soc-lab directory. The script was downloaded manually to Windows to generate activity for analysis.

```
$scriptPath = Join-Path $env:TEMP 'simulation.ps1'
Invoke-WebRequest -Uri 'http://192.168.50.20:8080/simulation.ps1' -OutFile $scriptPath -UseBasicParsing
```

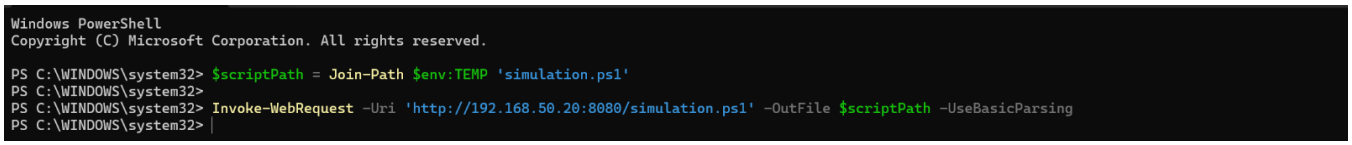


Figure 2. Download command for simulation.ps1 from the lab HTTP server.

The initial attempt to execute the script was blocked by the PowerShell execution policy. It was then launched with the following command:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File "$scriptPath"
```

The lab console displayed a completion message. This is separate from the SIEM investigation: the collected Sysmon events did not independently establish that sample.txt was downloaded.



Figure 3. Initial execution policy block followed by successful simulation execution.

Investigation method

The initial search used the known script name: index=main host=vmwin11 "simulation.ps1". This searches events containing the phrase; it does not restrict results to process creation events.

The field extraction generated through the Splunk interface did not reliably match all observed event structures. Explicit rex expressions were used to extract XML fields, and table displayed the relevant values. Example queries are included at the end of this report.

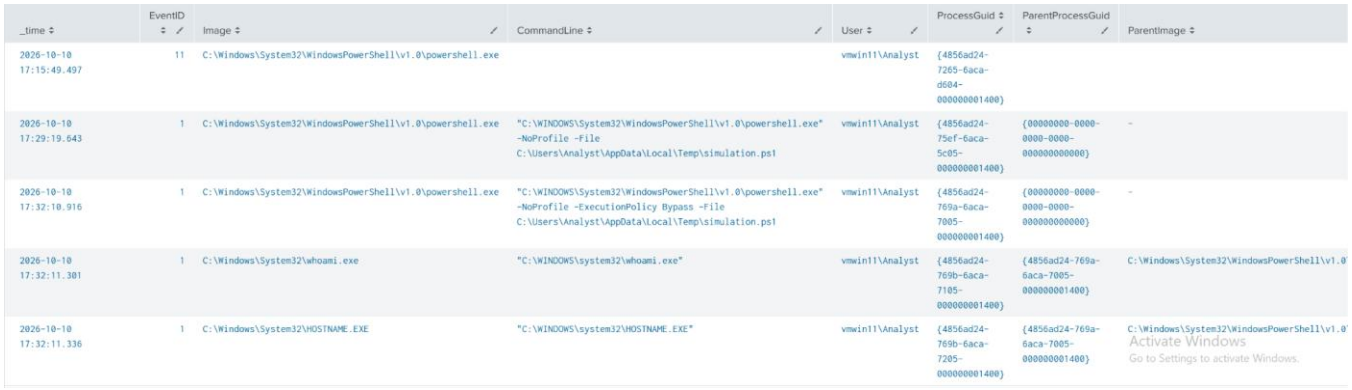
Process and network findings

On 10 October 2026 at 17:32:10.916, user vmwin11\Analyst launched PowerShell to execute C:\Users\Analyst\AppData\Local\Temp\simulation.ps1 with -NoProfile and -ExecutionPolicy Bypass. The process had PID 2984.

PowerShell ProcessGuid: {4856ad24-769a-6aca-7005-000000001400}. Matching this value with the child events' ParentProcessGuid confirmed that PowerShell launched whoami.exe and hostname.exe.

Displayed time	Event ID	Finding
17:32:10.916	1	PowerShell created, PID 2984
17:32:11.301	1	whoami.exe created as a child process
17:32:11.336	1	hostname.exe created as a child process
17:32:12.094	3	PID 2984 initiated TCP to 192.168.50.20:8080

Times above are displayed in Splunk for 10 October 2026. The display timezone was not independently verified; these values should not be treated as UTC without checking the corresponding UtcTime fields.



_time	EventID	Image	CommandLine	User	ProcessGuid	ParentProcessGuid	ParentImage
2026-10-10 17:15:49.497	11	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe		vmwin11\Analyst	{4856ad24-7265-6aca-d604-000000001400}		
2026-10-10 17:29:19.643	1	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell.exe" -NoProfile -File C:\Users\Analyst\AppData\Local\Temp\simulation.ps1	vmwin11\Analyst	{4856ad24-75ef-6aca-5c05-000000001400}	{00000000-0000-0000-0000-000000000000}	-
2026-10-10 17:32:10.916	1	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell.exe" -NoProfile -ExecutionPolicy Bypass -File C:\Users\Analyst\AppData\Local\Temp\simulation.ps1	vmwin11\Analyst	{4856ad24-769a-6aca-7005-000000001400}	{00000000-0000-0000-0000-000000000000}	-
2026-10-10 17:32:11.301	1	C:\Windows\System32\whoami.exe	"C:\WINDOWS\system32\whoami.exe"	vmwin11\Analyst	{4856ad24-769b-6aca-7105-000000001400}	{4856ad24-769a-6aca-7005-000000001400}	C:\Windows\System32\WindowsPowerShell\v1.0
2026-10-10 17:32:11.336	1	C:\Windows\System32\HOSTNAME.EXE	"C:\WINDOWS\system32\HOSTNAME.EXE"	vmwin11\Analyst	{4856ad24-769b-6aca-7205-000000001400}	{4856ad24-769a-6aca-7005-000000001400}	C:\Windows\System32\WindowsPowerShell\v1.0

Activate Windows
Go to Settings to activate Windows.

Figure 4. Process events showing the PowerShell ProcessGuid and matching child ParentProcessGuid values.

The network event recorded source IP 192.168.50.10, destination IP 192.168.50.20, destination port 8080, protocol tcp, and Initiated=true. Its PID matched the PowerShell process, and it occurred approximately 1.2 seconds after process creation on the same host.

This supports correlation with PowerShell, but the network event had a zero ProcessGuid and an unknown process image. PID reuse is possible, so PID alone is not a durable process identifier. Port 8080 and the connection event do not independently prove HTTP use or identify a downloaded file.

Logging limitations

Searching Splunk for sample.txt returned no results. A search of local Sysmon Event ID 11 records for the simulation date, performed with administrator privileges, also returned no matching file event.

The current Sysmon configuration was inspected with:

```
& "C:\Windows\Sysmon64.exe" -c
```

The FileCreate section used onmatch="include" and selected particular extensions and locations, including .ps1, .exe, .dll, and Downloads. The path C:\Users\Analyst\AppData\Local\Temp\SOC-Lab\sample.txt did not match those include conditions. This explains why that file would not be logged under the inspected configuration.

Absence of an event is not proof that the file was not downloaded. It means the collected telemetry could not establish that action. The investigation identified a gap between the simulation activity and the selected logging rules.

Recommended improvement

Add the following condition inside the existing FileCreate include section to cover files in the lab directory:

```
<TargetFilename condition="contains">\SOC-Lab\</TargetFilename>
```

Applying this rule would affect future events; it cannot reconstruct missing historical records. Successful application and end-to-end effectiveness were not verified in this report because the simulation was not repeated.

Conclusion

The investigation confirmed PowerShell execution and its whoami.exe and hostname.exe child processes through matching process GUIDs. A TCP connection was correlated using the host, PID, and timing, with limitations caused by missing process identity fields.

The download of sample.txt was not confirmed by the collected logs. The activity was known to be a controlled simulation. In a real investigation, these behaviors would require contextual validation because PowerShell, execution-policy bypass, whoami.exe, and hostname.exe can also be used legitimately.

No containment or remediation of an infected endpoint was performed. The practical outcome was reconstruction of observed activity and identification of a file creation logging gap.

Example investigation queries

Use a time range covering the simulation date. These queries extract fields from the single-quoted XML format observed in this lab.

Process execution and child relationships

```
index=main host=vmwin11 "simulation.ps1"
| rex "<EventID>(?<EventID>\d+)</EventID>"
| rex "<Data Name='Image'>(?<Image>[^<]*)</Data>"
| rex "<Data Name='CommandLine'>(?<CommandLine>[^<]*)</Data>"
| rex "<Data Name='User'>(?<User>[^<]*)</Data>"
| rex "<Data Name='ProcessGuid'>(?<ProcessGuid>[^<]*)</Data>"
| rex "<Data Name='ParentProcessGuid'>(?<ParentProcessGuid>[^<]*)</Data>"
| rex "<Data Name='ParentImage'>(?<ParentImage>[^<]*)</Data>"
| sort 0 _time
| table _time EventID Image CommandLine User ProcessGuid ParentProcessGuid ParentImage
```

Network activity associated with the observed PID

```
index=main host=vmwin11 "192.168.50.20"
| rex "<EventID>(?<EventID>\d+)</EventID>"
| rex "<Data Name='ProcessId'>(?<ProcessId>[^<]*)</Data>"
| search ProcessId=2984
| rex "<Data Name='Protocol'>(?<Protocol>[^<]*)</Data>"
| rex "<Data Name='Initiated'>(?<Initiated>[^<]*)</Data>"
| rex "<Data Name='SourceIp'>(?<SourceIp>[^<]*)</Data>"
| rex "<Data Name='DestinationIp'>(?<DestinationIp>[^<]*)</Data>"
| rex "<Data Name='DestinationPort'>(?<DestinationPort>[^<]*)</Data>"
| table _time EventID ProcessId Protocol Initiated SourceIp DestinationIp DestinationPort
```

Search for file evidence

```
index=main host=vmwin11 "sample.txt"
| rex "<EventID>(?<EventID>\d+)</EventID>"
| rex "<Data Name='TargetFilename'>(?<TargetFilename>[^<]*)</Data>"
| rex "<Data Name='ProcessGuid'>(?<ProcessGuid>[^<]*)</Data>"
| table _time EventID ProcessGuid TargetFilename
```

The file evidence query returned no results. These are investigation queries, not an implemented or validated alert rule.

What I learned

During this project, I learned how to connect facts from different Sysmon events and follow a PowerShell investigation in Splunk. I practised reconstructing a timeline, matching ProcessGuid with ParentProcessGuid to trace child processes, and assessing a network connection using the host, PID, and time.

I also learned how to configure Splunk and the Universal Forwarder to collect Windows Sysmon logs, troubleshoot collection issues, and check which events the Sysmon configuration covers. I practised creating and testing regular expressions with rex to extract XML fields and presenting the results with table.

The investigation taught me to separate confirmed observations from assumptions: a network connection does not by itself prove a file download, and missing logs do not prove that an action did not occur.